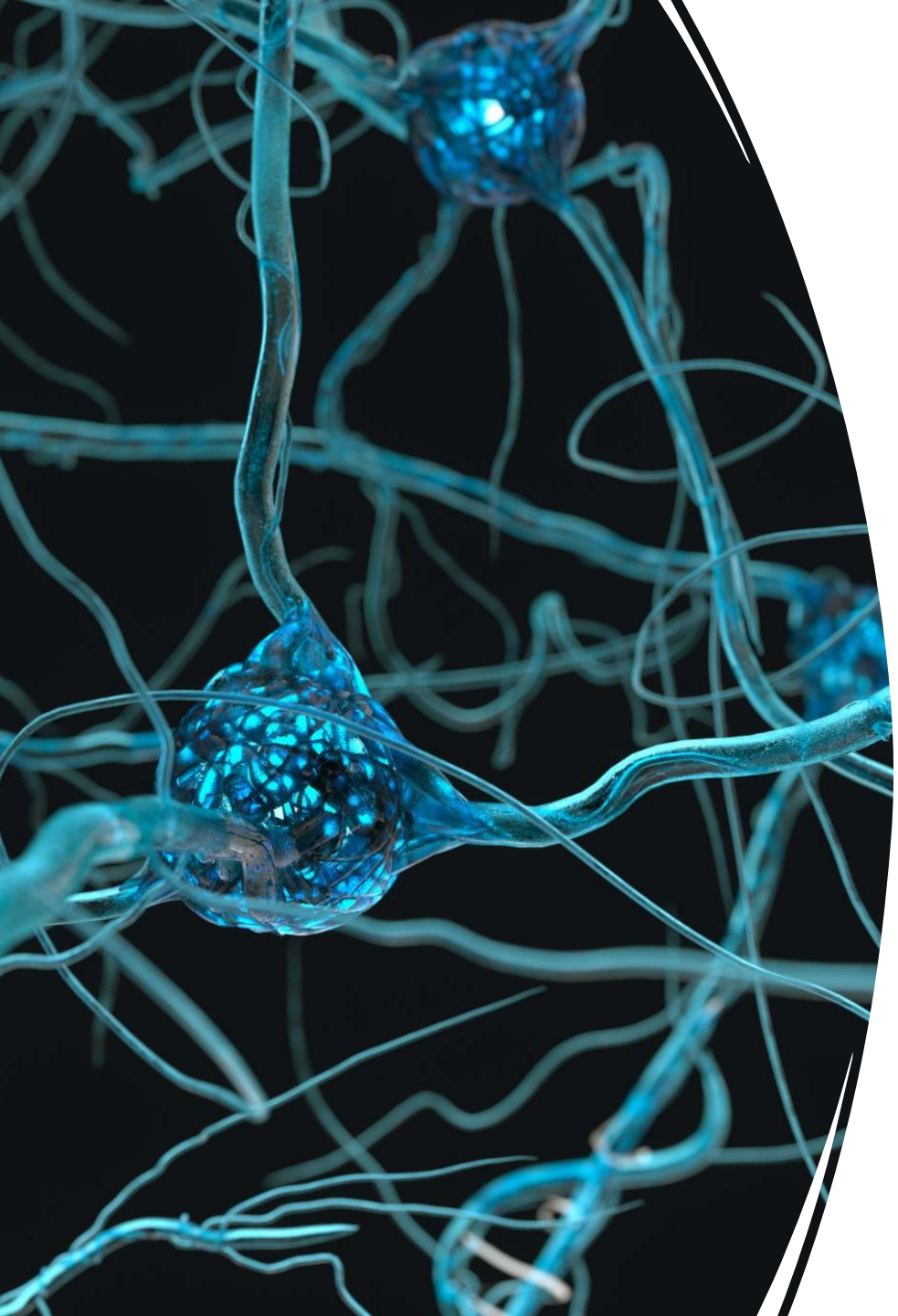

Functional and Partial Dependency

Objectives

- Review definitions and purpose of functional and partial dependency.
- Provide examples with explanations about partial and functional dependency.
- Demonstrate knowledge in identifying partial and functional dependencies in a given example.

Functional Dependency

- In a database table, **functional dependency** means that **one column's value depends on another column**.

- If you know the value of one column, you can figure out the value of another column.

Functional Dependency

Functional dependency helps us:

- Avoid data repetition
- Design better tables
- Move toward database normalization (like 2NF, 3NF)

Examples

- Each **StudentID** is **unique** to one student.
- If you know the **StudentID**, you can find out the **Name** and **Course**.

StudentID	Name	Course
1001	Alice	CS
1002	Bob	IT
1003	Charlie	CS

```
StudentID → Name
StudentID → Course
```

This means:

- StudentID functionally determines Name
- StudentID functionally determines Course

What is Partial Dependency?

- A partial dependency happens when:
- A column depends on part of a composite primary key, not the whole key.

StudentID	CourseID	StudentName	CourseName
1001	CS101	Alice	Computer Science
1002	IT202	Bob	Information Tech

Here the primary key is:
(StudentID, CourseID)

Why?
(because one student can take many courses, and one course can have many students)

Explanation:

- StudentName depends only on StudentID
- CourseName depends only on CourseID
- They don't depend on both StudentID and CourseID — only part of the key.
- ***That’s a partial dependency.***

Why is this a problem?

- It causes repeated data
- Wastes storage
- Makes updates and deletes more risky (update anomalies)

Explanation:

- Solution:
- Break the table into smaller ones to remove the partial dependency.

Example 2:

Table name: StudentGrades

StudentID	CourseID	StudentName	CourseName	Grade
1001	CS101	Alice	CompSci	A
1001	IT201	Alice	InfoTech	B+
1002	CS101	Bob	CompSci	A-

Primary Key = (StudentID, CourseID)

Partial Dependencies:
- **StudentName** depends only on StudentID
- **CourseName** depends only on CourseID

Why is it a problem?
- StudentName is repeated for the same StudentID
- CourseName is repeated for the same CourseID

Example 2

Why not use StudentID as the primary key?

- Because one student can enroll in multiple courses.

If we use only StudentID as the primary key:

- The table wouldn't allow more than **one row per student**
- Trying to insert the second course for Alice would cause a **duplicate key error**

Example 3

- Table: OrderDetails

OrderID	ProductID	CustomerName	ProductName	Quantity
O001	P100	John	Mouse	2
O001	P101	John	Keyboard	1
O002	P100	Sarah	Mouse	1

Primary Key = (OrderID, ProductID)

Partial Dependencies:
- **CustomerName** depends only on OrderID
- **ProductName** depends only on ProductID

Why is it a problem?
- You repeat the customer name for each product in the same order.
- You repeat product names for each order that contains them.

Why not use OrderID as the primary key?

- Because a single order can contain multiple products.

Order O001 has:

- Product P100 (Mouse)

- Product P101 (Keyboard)

- If OrderID were the primary key:

- You could only store one product per order

- Trying to insert more products under the same OrderID would cause a duplicate key error

Example 4

- Table name: EmployeeProjects

EmpID	ProjectID	EmpName	ProjectName	Role
E001	PRJ01	Carla	Website Dev	Developer
E001	PRJ02	Carla	App Dev	Team Lead
E002	PRJ01	Alex	Website Dev	Tester

Primary Key = (EmpID, ProjectID)

Partial Dependencies:
- **EmpName** depends only on EmpID
- **ProjectName** depends only on ProjectID

Example 4

Why not use EmpID alone as the primary key?

- Because one employee can work on multiple projects.
- E001 | PRJ01 | Carla | Website Dev | Developer
- E001 | PRJ02 | Carla | App Dev | Team Lead
- Carla (EmpID = E001) is working on two different projects.
- If we use only EmpID as the primary key, the database would not allow two rows with the same EmpID.

Example 4

- That would cause:
- Duplicate primary key error
- You'd be unable to store multiple projects per employee

So we use a composite key:

- Primary Key = (EmpID, ProjectID)

1. Identify the functional dependency

EmpID	EmpName	Department
E001	Alice	HR
E002	Bob	IT
E003	Charlie	Finance

2. Identify the partial dependency and convert to 2NF

BookID	AuthorID	BookTitle	AuthorName
B001	A001	DB Systems	John Smith
B001	A002	DB Systems	Jane Doe
B002	A001	Networks 101	John Smith